

WEEKLY PROGRESS REPORT

Week 2: DNS Tunneling - Lab Deployment & AI Detection Engine

Lê Quý Tôn

April 12, 2026

Project: CyberSec Internship - Cygnus-Security
Related Issue: #20

1. Contribution Scope

This week's work contributes to the practical deployment phase of the Intrusion Detection System (IDS) project. It involves constructing an isolated enterprise network simulation and advancing the detection engine with Machine Learning (AI) capabilities.

2. Objective of This Week

- Deploy a 3-VM segmented laboratory environment (Victim, Gateway, Attacker).
- Simulate an active DNS tunneling attack using `dnsmasq` as a DNS trap and `dnscat2`.
- Develop an AI-driven, real-time DPI engine using an ID3 Decision Tree classifier.
- Implement active network mitigation utilizing dynamic OS-level `iptables` rules.

3. Detailed Work

3.1 Description

Constructed a controlled lab to validate DNS tunneling mechanics. The environment enforces strict Layer 3 network segmentation, forcing the Victim (Debian) to route DNS queries through the Gateway (Ubuntu), which forwards malicious packets to the Attacker (Kali).

3.2 Design / Method

Network Design: Layer 3 routing with NAT masquerading to mimic real enterprise infrastructure.

Detection Design: A hybrid model combining real-time log parsing (tailing Suricata's `eve.json`) and Machine Learning. The ML model is trained on a baseline dataset comprising Subdomain Length and Shannon Entropy scores.

3.3 Implementation

- Configured `dnsmasq` on the Gateway to act as a conditional forwarder (DNS Trap).
- Executed `dnscat2` without encryption to allow deep packet visibility.
- Developed the ML logic in `dns_entropy_analyzer.py` using Python's `scikit-learn` library.

4. Results / Output

- **Attack Simulation:** Successfully established an unencrypted reverse shell and executed simulated file exfiltration via TXT, CNAME, and MX records.
- **Detection Engine:** The AI model successfully distinguishes Base64-encoded malware from legitimate traffic patterns.
- **Documentation:** Created a comprehensive Standard Operating Procedure (SOP) for lab deployment.

5. Analysis

The simulation perfectly mimics how enterprise DNS servers act as couriers for malicious payloads. By utilizing an ID3 Decision Tree, the AI engine can mathematically and autonomously classify high-entropy strings, significantly reducing the manual overhead of analyzing raw DNS traffic. The shift from passive monitoring to active mitigation (dropping forwarded traffic via `iptables`) proves the viability of automated defense mechanisms.

6. Issues & Fixes

- **Issue:** Port 53 collision on the Ubuntu Gateway preventing `dnsmasq` deployment.
- **Fix:** Disabled and stopped the default `systemd-resolved` service and removed `/etc/resolv.conf`.
- **Issue:** `dnscat2` encryption crashes due to deprecated Fixnum/Bignum in modern Ruby environments.
- **Fix:** Bypassed the crash and ensured DPI visibility by running the server with `--security=open` and the client with `--no-encryption`.

7. Reusable Content ★

(Standard Operating Procedure: DNS Tunneling Lab Environment Setup)

1. Architecture Overview

The laboratory simulates an enterprise network undergoing a DNS tunneling attack, consisting of three distinct VMs:

- **Gateway (Ubuntu Server):** Acts as the central router, NAT provider, and DNS Forwarder.

- **Victim (Debian 12):** The compromised internal endpoint.
- **Attacker (Kali Linux):** The external C&C Server.

2. Network & Routing Configuration

To simulate realistic Layer 3 routing, IP Forwarding is enabled on the Gateway. A NAT masquerade rule is applied to provide internet access to the internal network. On the Attacker, a static route is configured to ensure response packets route back to the internal LAN via the Gateway.

3. DNS Trap Deployment

The Gateway is configured using `dnsmasq` to resolve normal traffic while forwarding malicious queries. The configuration dictates that clean traffic routes to public resolvers, while specific malicious domains forward all query types directly to the Attacker.

4. Malware Execution

The covert channel is established using `dnscat2`. This setup successfully bypasses Layer 3 restrictions via Layer 7 DNS Tunneling.

8. Next Step

- Integrate the Python script directly into the Gateway to process live Suricata traffic during an active `dnscat2` session.